

Crônicas do Espaço: Planejamento Algorítmico de Missões

A humanidade está iniciando uma nova era de exploração espacial. Diferentes agências e organizações estão planejando missões para explorar corpos celestes, estabelecer rotas de viagem e identificar os destinos mais promissores para investigação científica.

Para apoiar esse processo, deverá ser desenvolvido um **Sistema de Gerenciamento e Planejamento de Missões Espaciais**.

O sistema deverá trabalhar com informações obtidas a partir de **APIs públicas**, que servirão como fonte inicial dos dados utilizados pela aplicação.

A proposta é permitir que diferentes grupos trabalhem com diferentes universos e fontes de dados. Assim, o sistema poderá representar tanto o **universo real** quanto **universos fictícios**, desde que a fonte escolhida disponibilize dados por meio de uma API adequada ao projeto.

Algumas possibilidades são:

- dados reais do Sistema Solar;
 - *Star Wars*;
 - *Rick and Morty*;
 - ou outros universos fictícios, mediante disponibilidade de dados via API.
-

2. Objetivo

O objetivo do trabalho é desenvolver um sistema capaz de **consultar, organizar e utilizar informações sobre objetos, locais, personagens, veículos ou outros elementos de um universo temático escolhido**, empregando estruturas de dados e algoritmos estudados na disciplina.

O trabalho será desenvolvido em duas partes.

Nesta primeira parte, o grupo deverá:

1. selecionar uma fonte de dados;
2. consumir uma API;
3. estruturar os dados obtidos;
4. implementar **uma das três estruturas de dados avançadas estudadas na disciplina**;
5. integrar a estrutura escolhida ao sistema;
6. resolver um problema de seleção/otimização utilizando uma estratégia gulosa.

As demais estruturas serão implementadas na segunda parte do trabalho.

3. Escolha do universo e da API

Cada grupo deverá escolher um universo para o desenvolvimento do sistema.

O grupo poderá utilizar:

Universo real

Por exemplo, a [API The Solar System OpenData](#), que disponibiliza informações sobre planetas, luas, planetas anões, asteroides e outros corpos, incluindo características físicas e orbitais.

Universo fictício

Por exemplo, a [Star Wars API \(SWAPI\)](#), que disponibiliza informações sobre planetas, personagens, espécies, naves, veículos e filmes.

Outras APIs poderão ser utilizadas mediante aprovação.

Importante

A API escolhida deverá:

- disponibilizar os dados por meio de requisições HTTP;
- retornar dados estruturados, preferencialmente em JSON;
- possuir quantidade e diversidade de dados suficiente para as operações do trabalho;
- permitir acesso de forma compatível com a implementação desenvolvida pelo grupo.

O grupo deverá documentar a API utilizada, incluindo:

- nome da API;
- endereço;
- recursos/endpoints utilizados;
- exemplos de requisições;
- estrutura dos dados retornados;
- data da consulta.

A utilização de uma API diferente das sugeridas é permitida e incentivada, desde que a fonte forneça dados suficientemente ricos para a realização das operações solicitadas no trabalho.

4. Módulo de aquisição e organização dos dados

O sistema deverá contar com um módulo dedicado à obtenção e organização dos dados a partir da API selecionada.

As atribuições deste módulo incluem:

- efetuar as requisições necessárias;
- processar e interpretar as respostas recebidas;
- filtrar os dados essenciais;
- mapeá-los para as estruturas de dados internas da aplicação.

Com o objetivo de contemplar diferentes contextos e assegurar o cumprimento do cronograma, a avaliação **exclusivamente deste módulo** considerará duas modalidades de integração:

- **Nível 1 - Consumo Dinâmico (100% da pontuação do módulo de aquisição):** O programa executa requisições HTTP (**GET**) em tempo de execução diretamente na API, realiza o *parsing* do retorno (como JSON) dinamicamente e carrega os dados nas estruturas. **Não há obrigatoriedade de efetuar todas as buscas do sistema por meio de chamadas diretas à API.** Uma vez obtidas, as informações devem ser armazenadas e organizadas localmente para viabilizar as operações subsequentes, garantindo o tratamento adequado dos dados retornados.
- **Nível 2 - Consumo Estático (70% da pontuação do módulo de aquisição):** Não são feitas requisições de rede enquanto a aplicação é executada. A coleta prévia dos dados é feita externamente pelo grupo (utilizando Postman, navegador ou scripts de apoio), salvando o conteúdo bruto em arquivos locais (**.json** ou **.csv**). O sistema principal limita-se a ler e extrair as informações a partir desses arquivos locais.

Nota: A opção pelo Nível 2 impacta apenas a nota referente à etapa de aquisição de dados, sem qualquer prejuízo na avaliação das estruturas de dados (Hash, Trie, Árvore B) ou do algoritmo guloso.

5. Estrutura de dados

O sistema deverá disponibilizar os módulos necessários para trabalhar com as seguintes estruturas:

- **Tabela Hash**
- **Trie**

- **Árvore B**

Nesta primeira parte, o grupo deverá implementar completamente **APENAS UMA** dessas três estruturas. As outras duas estruturas não deverão ser implementadas neste momento, mas deverão ter suas respectivas interfaces projetadas pelo grupo, considerando as operações que serão necessárias ao sistema. Essas interfaces serão utilizadas como base para a implementação das estruturas na Parte 2. As interfaces deverão ser projetadas de forma a separar a utilização da estrutura de sua implementação, permitindo que o restante do sistema utilize a estrutura sem depender de seus detalhes internos.

O grupo deverá justificar a sua escolha considerando as operações realizadas pelo módulo desenvolvido e as características da estrutura selecionada.

5.1. Autoria, Implementação Própria e Código Instrumentado

O objetivo desta etapa não é a memorização de sintaxe, mas a compreensão profunda dos mecanismos internos de cada estrutura de dados.

O grupo pode consultar implementações de referência na literatura acadêmica ou na internet, porém, **é terminantemente proibido utilizar estruturas de dados nativas de alto nível (como `HashMap` do Java, `std::map` do C++, ou `dict` do Python)** para substituir a estrutura avançada escolhida para a Parte 1. A estrutura deve ser codificada no projeto.

Para comprovar o domínio sobre o código e evitar o uso de algoritmos como "caixas pretas", a estrutura implementada deverá ser obrigatoriamente **instrumentada**. Isso significa que o grupo deve injetar código no núcleo da estrutura para rastrear métricas internas de execução. As métricas deverão ser produzidas pela própria estrutura implementada, não estimadas ou calculadas posteriormente a partir dos dados de entrada.

Dependendo da estrutura escolhida, o sistema deverá rastrear e imprimir na tela:

- **Se Tabela Hash:** O número exato de colisões ocorridas durante o carregamento dos dados da missão e o fator de carga (*load factor*) atual da tabela.
- **Se Trie:** A quantidade exata de nós visitados/percorridos durante operações de busca de elementos ou pesquisa por prefixos.
- **Se Árvore B:** O número total de operações de *split* (divisão de nós) e a altura final da árvore após o carregamento inicial dos dados.

6. Funcionalidades do sistema

O sistema deverá disponibilizar funcionalidades para consulta e gerenciamento dos elementos do universo escolhido. As operações adicionais deverão fazer uso efetivo da estrutura de dados escolhida sempre que pertinente.

As funcionalidades deverão envolver, no mínimo:

6.1. Consulta de elementos

Permitir localizar elementos do universo a partir de alguma forma de identificação.

Exemplos:

- localizar um planeta;
- localizar uma nave;
- localizar um personagem;
- localizar uma espécie;
- localizar um corpo celeste.

6.2. Pesquisa

Permitir realizar pesquisas utilizando atributos relevantes dos dados obtidos. Por exemplo:

- nome;
- tipo;
- categoria;
- localização;
- características físicas;
- características da nave;
- espécie;
- ou outros atributos disponíveis na API escolhida.

6.3. Listagem e filtragem

O sistema deverá permitir gerar listagens de elementos que satisfaçam determinados critérios.

Os critérios deverão ser definidos pelo grupo de acordo com o universo escolhido.

6.4. Operações adicionais

O grupo deverá implementar outras operações relevantes ao universo escolhido.

A funcionalidade deverá ser justificada na documentação do trabalho.

7. Planejamento de uma missão

Considerando os desafios inerentes à exploração espacial, o sistema deverá resolver um problema de otimização utilizando uma estratégia gulosa. O grupo deve escolher apenas uma das duas opções abaixo para implementar nesta primeira parte:

Opção A: Planejamento e Triagem de Missões (Otimização Logística)

Uma agência de exploração possui recursos limitados para suas missões. O sistema deverá selecionar um conjunto de destinos que maximize o retorno da missão, atuando como um filtro inteligente de prioridades.

Cada elemento que pode ser visitado ou explorado deverá possuir características que permitam determinar seu custo e seu benefício para a missão. Por exemplo, no Sistema Solar, poderiam ser considerados: distância, massa, gravidade, temperatura, tipo do corpo, dificuldade de exploração ou potencial científico. O problema deverá envolver pelo menos **duas restrições/recursos** ou uma quantidade limitada de algum recurso, e a solução deverá tomar decisões sucessivas de seleção de candidatos. O grupo deverá:

1. Definir o critério utilizado para selecionar os candidatos.
2. Implementar a estratégia gulosa.
3. Apresentar a solução encontrada.
4. Justificar a escolha do critério.
5. Discutir as limitações da estratégia utilizada (avaliando cenários em que a escolha estritamente gulosa pode falhar em encontrar a solução ótima global).

Opção B: Comunicação Interplanetária e Telemetria (Compressão de Dados)

A largura de banda para comunicação entre naves, sondas e a base de controle na Terra é um recurso crítico e extremamente limitado. Transmitir dados em texto puro consome tempo de antena e energia inviáveis para a agência. O sistema deverá utilizar o Algoritmo de Huffman (uma estratégia gulosa de compressão sem perdas) para otimizar o envio dessa telemetria.

O módulo deverá receber e comprimir uma string longa de dados extraída do universo escolhido (por exemplo, os dados consolidados de um planeta específico, relatórios de naves, ou o payload bruto da API).

O grupo deverá:

1. Calcular e exibir a tabela de frequências dos caracteres do texto processado.
2. Construir e demonstrar a estrutura da Árvore de Huffman gerada.
3. Exibir o dicionário de códigos (o mapeamento exato de cada caractere original para a sua nova representação em bits).
4. Apresentar a taxa de compressão final (comparando o tamanho estimado em bits da string original não comprimida com o tamanho exato da string após a compressão).

8. Requisitos mínimos da Parte 1

Ao final desta etapa, o sistema deverá possuir:

- integração com uma API;
 - módulo de aquisição dos dados;
 - armazenamento local dos dados obtidos;
 - implementação completa de **uma** entre Hash, Trie e Árvore B;
 - integração da estrutura escolhida com o sistema;
 - operações de consulta/pesquisa;
 - funcionalidades de filtragem/listagem;
 - definição de um problema de seleção de destinos;
 - implementação de uma das estratégias/problemas gulosos definidos na Seção 7.
 - justificativa das decisões de projeto.
-

9. Documentação

O grupo deverá entregar documentação contendo, no mínimo:

Fonte de dados

- API escolhida;
- justificativa da escolha;
- endpoints utilizados;
- exemplos de requisições;
- estrutura dos dados obtidos.

Modelagem

- representação dos elementos;
- atributos utilizados;
- operações implementadas;
- decisões de projeto.

Análise

Será necessário selecionar um procedimento específico da estrutura adotada — tais como o re-hashing na **Tabela Hash**, a partição de nós na **Árvore B** ou o crescimento dinâmico na **Trie** — para a realização de uma **análise detalhada de complexidade amortizada**. O relatório deve fundamentar, por meio de justificativa matemática ou pela aplicação dos métodos contábil ou do potencial, a razão pela qual a métrica tradicional de pior caso $O(n)$ apresenta um cenário demasiadamente conservador frente ao custo médio obtido durante uma sucessão prolongada de requisições na aplicação.

Estrutura de dados

- estrutura escolhida;
- justificativa da escolha;
- operações implementadas;

- análise da complexidade das principais operações;
- integração com o sistema.

Algoritmo guloso escolhido

- definição do problema;
 - estratégia adotada;
 - justificativa do critério de escolha;
 - análise da solução produzida.
-

10. Entrega e Avaliação

A avaliação da Parte 1 será composta pela análise técnica dos artefatos submetidos e por uma validação prática presencial, garantindo a autoria e o domínio conceitual do grupo.

10.1. Entregáveis (Repositório e Relatório)

O grupo deverá entregar via repositório Git (GitHub ou GitLab), compartilhado com a docente até a data limite estipulada:

- **Código-fonte:** O código completo, modularizado e funcional.
- **Documentação:** O relatório consolidado (em arquivo README.md ou .pdf na raiz do repositório) contendo todas as exigências da Seção 10, incluindo a origem dos dados, decisões de projeto e, criticamente, a Análise Amortizada.

10.2. Avaliação: Arguição em Laboratório (Check de Bancada)

Para otimizar o tempo e focar na construção do software, a defesa do trabalho não ocorrerá no formato de seminário tradicional em sala de aula. A avaliação será realizada através de um "Check de Bancada" durante aula prática de laboratório do dia **02/10/2026**.

Os avaliadores passarão avaliando cada grupo (com duração estimada de 5 a 7 minutos). Durante a arguição, o grupo deverá estar preparado para:

1. Execução: Compilar e executar o sistema no próprio ambiente de desenvolvimento em tempo real.
2. Demonstração de Instrumentação: Acionar as funcionalidades que disparam a estrutura avançada (Hash, Trie ou Árvore B) e mostrar as métricas instrumentadas sendo impressas no terminal (ex: contagem de colisões, nós visitados ou operações de split).
3. Validação Gulosa: Executar a estratégia gulosa escolhida (Planejamento da Missão ou Compressão de Huffman) e explicar rapidamente o resultado gerado.

4. **Comprovação de Autoria:** Responder a perguntas diretas sobre as estruturas, justificar blocos lógicos específicos ou realizar alterações pontuais no código a pedido da docente.

Observação: A arguição é individual. Caso um integrante não consiga explicar a lógica do código entregue, demonstrar como os dados estão sendo manipulados em memória ou justificar a escolha estrutural, a sua nota individual tenderá a zero, independentemente da qualidade do repositório entregue.

Parâmetros Logísticos e de Avaliação

- **Formação de Grupos:** O trabalho poderá ser desenvolvido individualmente, em duplas ou em trios. Recomenda-se fortemente a divisão clara de responsabilidades (ex: ingestão de dados, codificação da estrutura avançada e elaboração do algoritmo guloso), mas **todos** os integrantes devem compreender o funcionamento global do sistema para a arguição individual.
- **Ponderação na Nota Semestral:** O projeto prático como um todo representa 25% da nota final da unidade curricular, com o esforço e a avaliação distribuídos da seguinte forma:
 - Parte 1 (Prototipagem e Design): 10% da nota;
 - Parte 2 (Implementação Completa e Integração): 15% da nota.
- **Prazo de Entrega:** O repositório Git com o código-fonte e o relatório técnico deverá estar finalizado até ao dia **02/10/2026 às 10h**. A avaliação prática no formato Check de Bancada decorrerá na aula de laboratório que ocorrerá no mesmo dia à tarde.